PRESIDENCE DU CONSEIL

MINISTERE DE LA PLANIFICATION DU DEVELOPPEMENT ET DE LA COOPERATION



Institut Africain d'Informatique
Etablissement Inter-Etats d'Enseignement Supérieur
Représentation du Togo (IAI-TOGO)
07 **BP** 12456 Lomé 07 **Tél.** : (+228) 22 20 47 00
E-mail : iaitogo@iai-togo.tg **Site Web**: www.iai-togo.tg

Support de cours

Langage C

Chargé de cours : M. **AKOMAGBE** Innocent

Dernière mise à jour : 18/01/2026

56 68 78 6F 76 20 66 68 78 61 20 74 78 6C 20 76 68 20 75 6C 76 74
78 68 75 72 71 77 20 64 20 73 68 78 77 2D 68 77 75 68 20 64 6F 6F
68 75 20 77 75 72 73 20 6F 72 6C 71 20 76 64 78 75 72 71 77 20 6D

Année Académique 2025 - 2026

Sommaire

Sommaire 1
Chapitre 1 : Présentation du langage C 3
I. Historique du langage C 3
II. Caractéristiques du langage C 4
III. Domaines d'utilisation du langage C 4
IV. Outils 4
Chapitre 2 : Notions de base 6
I. Structure et syntaxe générale 6
II. Les unités lexicales 10
Chapitre 3 : Les variables et les constantes 12
I. Les identificateurs 12
II. Les variables et les constantes 12
III. Les types 14
IV. Notation 16
Chapitre 4 : Les entrées-sorties 18
I. Introduction 18
II. Les sorties 18
III. Les entrées 22
Chapitre 5 : Les opérateurs 29
I. Introduction 29
II. Les opérateurs 29
Chapitre 6 : Les structures conditionnelles 39
I. Introduction 39
II. Les structures if 39
III. L'instruction `switch` 42
IV. L'opérateur conditionnel 44
Chapitre 7 : Les structures répétitives 45
I. Introduction 45
II. La boucle for 45
III. La boucle While 46
IV. La boucle `do…while` 47
V. `Choix` d'une boucle 48
Chapitre 8 : Les tableaux 49
I. Introduction 49
II. Notion de tableau 49
III. Les tableaux unidimensionnels 50

IV. Les tableaux multidimensionnels 52
Chapitre 9 : Les chaines de caractères 55
I. Notion de chaine de caractères 55
II. Déclaration et initialisation des chaines de caractères 55
III. Quelques fonctions de la librairie standard 57
Chapitre 10 : Les fonctions 58
I. La notion de fonction 58
II. Définition et appel d'une fonction 58
III. Arguments et durée de vie d'une variable 60
IV. Retour d'une fonction 62
V. La notion de récursivité 63
Chapitre 11 : Les structures 65
I. Notions de structures 65
II. Définition d'une structure 65
III. Définition de type avec `typedef` 67
Chapitre 12 : Les Énumérations 69
I. Notion d'énumération 69
II. Définition d'une énumération 69
III. Attribution de valeurs explicites 69
IV. Déclaration et utilisation d'une variable de type énumération 70
V. Définition d'une Énumération avec typedef 72
Chapitre 13 : Les unions 73
I. Définition d'une union 73
II. Différences clés entre une structure et une union 73
III. Utilisation d'une union dans une structure 74
Chapitre 14 : Introduction aux pointeurs 76
I. Notions de pointeur 76
II. Déclaration de pointeurs 76
III. Tableaux et Pointeurs 77
IV. Passage par adresse 77
Annexe 79
I. Résumé des types en C 79
II. Manipulation de caractères <ctype.h> 80
III. Fonction standard sur les chaînes de caractères dans <string.h> 81
IV. Résumé des opérateurs 83

I. Historique du langage C

Le langage C a été créé en 1972 par Dennis Ritchie dans les Laboratoires Bells comme pour objectif principal : La réécriture du système d'exploitation UNIX. Grace aux nombreux avantages qu'il offre, le langage C est devenu aujourd'hui l'un des langages de programmation les plus utilisés. Le langage C est directement dérivé du langage B originellement développé par Kenneth Thompson avec l'ajout de quelques améliorations, notamment l'ajout de types.

En 1978, Brian Kernighan coécrit avec Dennis Ritchie la première édition de l'ouvrage ***The C Programming Language*** qui décrit le langage de programmation C et était orientée vers les créateurs de compilateurs. Grace aux évolutions du langage et des compilateurs, une seconde édition a été publiée en 1988. Son succès a conduit à sa normalisation en 1989 par l'ANSI (American National Standard Institute). Cette norme est dite ANSI C ou C89 (formellement ANSI X3.159-1989). En 1990, cette norme a également été adoptée par l'Organisation internationale de normalisation (C90, C ISO). Ensuite, d'autres versions ont vu le jour dont le C99, le C11, le C17.

Une nouvelle version a été publié en octobre 2024 sous le nom C23 vient en remplacement au C17.

II. Caractéristiques du langage C

Les avantages qu'offre le langage C s'articulent autour des points suivants

- **Modularité :** Le programme peut être séparé en modules écrits et compilés séparément puis liés ensemble pour produire un programme exécutable ;
- **Facilité :** Il est facile à apprendre ;
- **Structuration :** Le C un langage de programmation structurée : Chaque programme est composé d'une ou de plusieurs fonctions ;
- **Efficacité :** Il produit des programmes efficients à cause de proximité avec le matériel. Il permet aussi l'écriture de code bas niveau ;
- **Portabilité :** Peut compiler pour une variété de plateformes et d'architectures.

III. Domaines d'utilisation du langage C

Le langage C est utilisé dans de nombreux domaines où la performance, la rapidité et le contrôle de la machine sont importants. Il est utilisé pour développer les logiciels comme : les systèmes d'exploitation, les compilateurs, les assembleurs, les éditeurs de textes, les pilotes réseaux, les bases de données, les jeux vidéo, etc.

Les logiciels développés peuvent avoir des champs d'application dans les domaines tels que : le domaine scientifique et médicale, etc.

IV. Outils

La pratique de la programmation demande de posséder un certain nombre d'outils permettant de rédiger et de traduire les instructions qui seront données à l'ordinateur.

Dans le cas du langage C, deux outils sont nécessaires : Le compilateur, l'éditeur de texte et accessoirement un débogueur.

1. L'éditeur de texte

Un éditeur de code source est un programme d'édition de texte spécialement conçu pour l'édition du code source d'un programme informatique. Il peut s'agir d'une application autonome ou intégrée à un Environnement de Développement Intégré (EDI ou IDE) ou à un navigateur Web.

Elle comporte des fonctionnalités spécialement conçues pour simplifier et accélérer la saisie du code source, telles que : la coloration syntaxique, l'indentation, la saisie semi-automatique, la correspondance d'accolades, etc.

Exemple d'éditeur de code C: Code Blocks, Vs Code, Dev C++, Visual Studio, Vim, Nano, etc.

2. Le Compilateur

Un compilateur est un logiciel qui lit et traduit un code source en code machine directement exécutable par le processeur de l'ordinateur. Généralement, le code source est écrit dans un langage de programmation (C dans notre cas).

Exemple de compilateur C: GCC, Clang, Microsoft Visual C++, Oracle C compiler.

Dans le cadre de ce cours, nous utiliserons comme éditeur de texte **Code Blocks** et **GCC** comme compilateur.

I. Structure et syntaxe générale

1. Premier programme

Notre premier programme effectuera une simple action qui consistera à afficher à l'écran le message « **Bonjour le monde !** ».

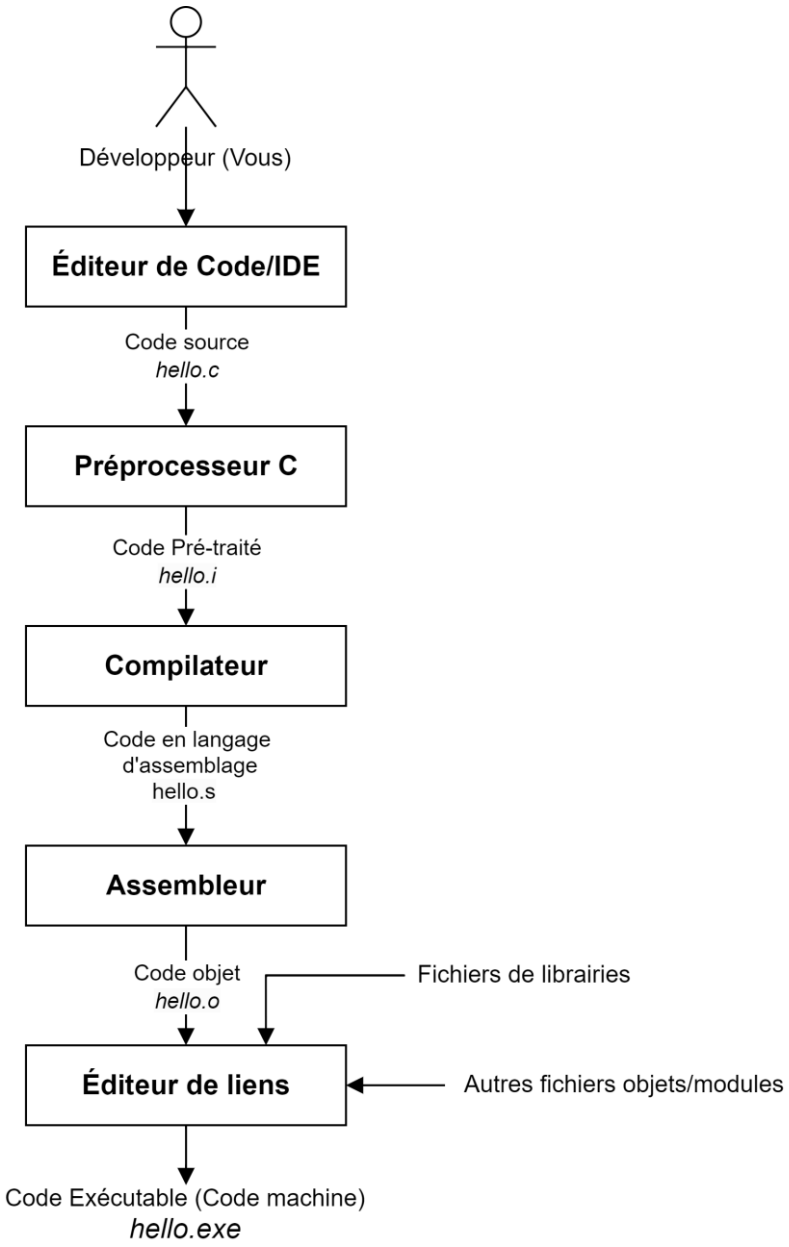
Dans le fichier `main.c` ouvert dans notre éditeur de texte, nous écrivons les lignes de code suivantes :

```c
#include <stdio.h>
int main()
{
    printf("Bonjour le monde !\n");
    return 0;
}
```

2. La compilation

La compilation d'un programme écrit en C comme dans la plupart des autres langages de programmation doit suivre un certain nombre d'étapes avant d'aboutir au fichier exécutable.

Le schéma suivant résume le processus de compilation d'un programme écrit en langage C.



Nous distinguons les étapes suivantes :

2.1. Le prétraitement

Il est effectué par le préprocesseur. Le préprocesseur est un programme qui effectue des transformations purement textuelles sur le code source. Il est le premier programme qui s'exécute et effectue les tâches telles que l'inclusion de fichiers, la définition de macros, la suppression des commentaires, etc.

2.2. Compilation

Le compilateur est un programme qui traduit le code source prétraité par le préprocesseur en langage assembleur.

2.3. Assemblage

Cette étape est à la charge de l'assembleur**.** L'assembleur est un programme qui traduit le langage assembleur en langage machine. Le langage machine est le langage que le microprocesseur comprend et peut exécuter.

2.4. Édition de liens

L'éditeur de liens est un programme qui combine les fichiers objets générés par le compilateur et l'assembleur pour créer un fichier exécutable.

3. Explication du premier programme

Reprenons notre premier programme précédemment écrit :

- `#include <stdio.h>`

Cette première partie est une instruction destinée à être traité par le préprocesseur. Cette instruction est traitée par le préprocesseur lors de la compilation et lui indique de trouver le fichier `stdio.h`, et d'inclure une copie du contenu de ce fichier à ce point du code.

Les fichiers avec l'extension **.**h sont des fichiers d'entête et peuvent contenir des prototypes de fonction, les déclarations de fonction. À chaque fois que nous avons besoin de la définition d'une fonction, nous devons inclure le fichier d'entête dans lequel ces fonctions sont déclarées.

Nous distinguons deux catégories de fichiers d'entête : **Les fichiers d'entête systèmes** et **ceux définis par l'utilisateur**. Les symboles <> autour des fichiers d'entête sont cherchés dans les fichiers d'entêtes systèmes et ceux entourés par les symboles **""** sont cherchés dans le dossier de travail courant.

Quelques fichiers d'entête provenant de la librairie standard de C :

- **`#include<stdio.h>`** : Il est utilisé pour les instructions d'entrée-sortie à l'aide des fonctions `printf()` et `scanf()`**.**

- `#include<stdlib.h>` : Elle déclare des fonctions qui effectuent la conversion de nombres, la gestion de la mémoire et d'autres tâches.
- `#include<math.h>` : Contient des fonctions mathématiques prédéfinies comme `sqrt()` et `pow()`.

- **`int main()`**

Cette ligne indique la définition de la fonction **`main`**. Cette fonction constitue le point d'entrée principale du programme, d'où son nom. Cette fonction est délimitée par les accolades «{» et «} ». Les instructions situées entre ces accolades forment un bloc.

- **L'instruction `printf`**

Cette instruction permet de faire un affichage continu à l'écran (de gauche à droite). Le caractère `\n` indique une fin de la ligne et permet de mettre fin à la ligne actuelle en renvoyant le curseur sur une nouvelle ligne.

Application : Modifiez le premier programme pour afficher les messages suivants sur trois lignes : *Bonjour à moi, Bonjour à toi et Bonjour à nous*.

- **L'instruction `return 0;`**

Elle permet à la fonction **`main`** d'envoyer un retour (`0` dans notre cas) afin d'indiquer au système d'exploitation que notre programme s'est bien exécuté et aucune erreur n'a été rencontré.

4. Syntaxe générale d'un programme en langage C

Un programme en langage C suit la syntaxe suivante :

Syntaxe :

```
[Commentaires/Déclaration]
<Directives préprocesseur>
<Section de définitions>
<Section de déclaration globale>
int main()
{
    [Déclaration de variable locales]
    [Corps de la fonction main]
    return 0;
}
[Définition de fonctions]
```

Exemple :

```c
/* Un programme d'exemple */
#include <stdio.h>
#define x 3
int main()
{
   int y = 2;
   printf("%d", x+y);
   return 0;
}
```

II. Les unités lexicales

1. Les mots-clés

Un mot-clé est un mot réservé pour la propre utilisation du langage de programmation. Ils ne peuvent donc pas être utilisés comme identificateurs ou dans des situations qui ne leur correspondent pas. Le langage C compte au total 32 mots-clés qui sont :

auto	return	void
while	register	sizeof
unsigned	goto	break
do	long	char
default	switch	if
case	struct	const
continue	short	union
for	static	else
float	int	signed
double	volatile	enum
extern	typedef	

2. Les commentaires

Comme tous les langages de programmation moderne, le langage C autorise les commentaires dans un programme. Ils servent à la documentation du programme et sont destinés aux lecteurs humains. Ils n'ont aucune incidence sur le code et sont totalement ignorés lors de la compilation.

Ils peuvent être placés à tout endroit d'un programme ou l'espace est autorisé et peuvent être sur une seule ligne comme sur plusieurs lignes.

Un commentaire sur une ligne commence par les caractères `//`.

Exemple :

```
// Ceci est un commentaire sur une ligne
```

Un commentaire sur plusieurs lignes est délimité par les caractères `/**/`

Exemple :

```
1. /* commentaire de la ligne 1
2.    commentaire de la ligne 2
3.    commentaire de la ligne 3
4. */
```

3. L'espace blanc

Comme dans le langage naturel humain, les mots sont séparés par un espace, un signe de ponctuation ou de fin de ligne afin de faciliter la lecture et la compréhension. La même chose s'applique également au langage C. L'espace blanc est imposé pour séparer des éléments adjacents comme les mots clés, les identificateurs et les constantes. Cependant, ils peuvent être utilisés dans les autres cas où ils ne sont pas exigés dans le but d'améliorer la lisibilité et la présentation d'un programme. Dans ces cas, ils seront ignorés par le compilateur.

Nous devons obligatoirement écrire `int x, y;` et non `intx,y;`. Toutefois, nous pouvons écrire `int x,y,z;` mais nous allons préférer écrire `int x, y, z;`.

I. Les identificateurs

Les identificateurs permettent de designer les différents objets manipulés par un programme : les variables, les constantes, les fonctions, etc. Comme dans la majorité des langages de programmation, ils sont formés d'une suite de caractères choisis parmi les lettres ou les chiffres. Cependant, un identificateur commence nécessairement par une lettre.

Comme dans la plupart des langages de programmation, les identificateurs doivent respecter un certain nombre de règles. Ces règles sont les mêmes que celles étudiées au cours d'Initiation à l'Algorithmique **:)**.

Concernant ces règles, notons que :

- Le caractère **`Underscore(_)`** est considéré comme une lettre. Il est donc autorisé à apparaitre au début, au milieu, à la fin, c'est-à-dire à n'importe quelle position dans un identificateur.
- Les majuscules et les minuscules sont autorisés, mais ne sont pas équivalentes : On dit que le langage C est sensible à la casse. Ainsi, en C, `resultat` et `Resultat` désignent deux objets différents.
- Il faudrait aussi éviter d'utiliser les caractères accentués dans le nom des identificateurs.

II. Les variables et les constantes

En informatique, une variable est une portion de mémoire, appelée `objet`, à laquelle nous donnons un nom (identificateur). Elle permet de stocker une valeur donnée. Si cette valeur peut être modifiée au cours du programme, nous parlons de variable, dans le cas contraire, nous parlons de constante.

1. Déclaration des variables

Ci-dessous la syntaxe de déclaration des variables en C :

Syntaxe de déclaration d'une variable :

```
<type> <identificateur>;
```

En C, il est possible d'initialiser les variables lors de leur déclaration.

Exemple :

```
1. int age;
2. char TAB = '\t';
3. float x = 1.028e-4;
```

Remarques :

- La ligne 1 déclare une variable nommée `age` de type `int`, c'est-à-dire qu'elle est destinée à contenir des nombres entiers (relatifs).
- La ligne 2 déclare une variable qui peut stocker un caractère. Dans notre cas, il s'agit du caractère tabulation.
- La dernière ligne déclare une variable qui est de type `float` et contient un nombre flottant (approximativement un nombre réel).

2. Déclaration des constantes

L'utilisation des constantes en programmation est vivement conseillée, car elles permettent :

- Une notation plus simple et plus lisible ;
- La possibilité de modifier simplement la valeur spécifiée dans la déclaration au lieu de rechercher les occurrences, puis de modifier chacune de ces occurrences dans le programme.

Pour déclarer une constante, nous pouvons utiliser la directive `#define` du préprocesseur. Cependant, dans les récentes normes du C, on peut utiliser le mot clé `const`.

Exemples :

```
1. #define PI      3.14159
2. #define TITRE   "Langage C"
3.
4. const float pi = 3.14;
5. const char letter = 'I';
```

Figure 2: Déclaration de constantes

Ces constantes peuvent être des constantes entières, réelles, caractère ou chaine de caractères.

Il faut noter que lorsque `#define` est utilisé, une substitution est effectuée par le préprocesseur avant la compilation. Ainsi, le compilateur ne voit que le code après que toutes les macros ont été remplacées, ce qui peut parfois mener à des erreurs si les macros ne sont pas correctement définies.

III. Les types

Le type d'une donnée nous indique l'ensemble des valeurs admissibles, le nombre d'octets à réserver en mémoire et l'ensemble des opérateurs qui peuvent être appliqués à un objet.

Les types de base du langage C se répartissent en trois grandes catégories en fonctions de la nature des informations qu'ils permettent de représenter :

- Les nombres entiers (`int`)
- Les nombres flottants (`float` ou `double`)
- Les caractères (`char`)

1. Les types entiers

Le langage C permet de représenter les entiers avec des tailles différentes, désignés par les mots-clés suivants :

- `short int` ou simplement `short`
- `int`
- `long int`
- `long long int` (introduit par la norme C99)

Ces entiers dépendent non seulement du mot-clé considéré, mais aussi de la machine utilisée puisque les entiers ne possèdent pas la même taille sur toutes les machines.

Face à ces incertitudes quant à la représentation des entiers, la norme C99 a introduit de nouveaux types entiers : les entiers de taille fixe. Ces types sont définis dans l'en-tête **<stdint.h>** et sont les suivants : **int8_t**, **uint8_t**, **int16_t**, ..., **uint64_t.**

Cependant, il est plus commun de trouver les types **short** avec une taille de 16 bits et les **int** codés sur 32 bits.

Par défaut, un type est signé, le mot-clé **`signed`** est donc implicite et facultatif, sauf pour le type **`char`**, car il sert à distinguer le type **`char`**, qui stocke des caractères, des types **`signed`** char et **`unsigned char`**, qui stockent des entiers. Les types non signés sont précédés du mot-clé **`unsigned`**.

Exemple : +533, 43, -274

2. Les types flottants

Les types flottants permettent de représenter, de manière approchée, une partie des nombres réels. Le langage C dispose de trois types de flottants avec des tailles différentes : **`float`**, **`double`** et le **`long double`**.

Comme dans la plupart des langages de programmation, les constantes flottantes peuvent s'écrire selon la notation décimale ou exponentielle.

- **La notation décimale :** Elle doit comporter obligatoirement un point (correspondant à la virgule). La partie décimale ou la partie décimale peut être omise.
  Exemples : `3.14` `-5.25` `-.25` `4.` `.27`
- **La notation exponentielle :** La lettre `e` ou `E` est utilisée pour représenter l'exposant.
  Exemples : `2.15E4` (eq. `21500.000000`), `2.3E-5` (eq. `0.000023`), `612.2E-33`

Toutes les constantes déclarées sont considérées par défaut comme des types **`double`** par le compilateur. Nous devons précéder ces valeurs de la lettre **`F`** ou **`f`** pour indiquer un type **`float`** ou par la lettre **`L`** ou **`l`** pour le type **`long double`**.

3. Les types caractères

Le langage C comme la plupart des langages de programmation, permettent de manipuler les caractères. Ces caractères sont généralement codés sur un octet en mémoire. Le code ASCII (American Standard Code for Information Interchange) est une norme qui définit l'encodage de ces caractères. Elle permet de représenter d'une part les caractères imprimables et non imprimables (tabulation, fin de ligne, retour à la ligne, etc.).

Le code ASCII ne permet de représenter qu'un nombre limité de caractères (128 ou 255 pour la table étendue). Afin de représenter un grand nombre de caractères, d'autres normes ont vu le jour, notamment l'Unicode qui permet de représenter un grand nombre de caractères.

IV. Notation

Les caractères imprimables du code ASCII (à l'exception des caractères `'`, `''`, `\`) se notent en les écrivant entre apostrophes (ou quotes).

Exemples : `'A','Z','E','!','?'`

Les caractères en exceptions précédemment cités doivent être précédés du caractère `\`

Exemples : `'\'','\'''','\\'`

Les caractères non imprimables doivent être notés comme suit :

Notation	Code ASCII (Hex)	Abréviation Usuelle	Signification
`\a`	07	BEL	Cloche ou bip (Alerte ou audible bell)
`\b`	08	BS	Retour arrière (Backspace)
`\f`	0C	FF	Saut de page (Form Feed)
`\n`	0A	LF	Saut de ligne (Line Feed)
`\r`	0D	CR	Retour chariot (Carriage Return)
`\t`	09	HT	Tabulation horizontale (Horizontal Tab)
`\v`	11	VT	Tabulation verticale (Vertical Tab)

Il est possible d'utiliser directement le code du caractère en l'exprimant après le caractère « antislash » ou « backslash » (\) soit sous forme octale ou sous forme hexadécimale précédée de x.

Exemples : `'\x07','\x0d','\15'`

Application :

```c
#include <stdio.h>
int main()
{
    char c;

    while(c = getchar()) {
        printf("%c\a\n", c);
    }

    return 0;
}
```

Figure 3: Exemple d'utilisation de quelques caractères non imprimable.

NB : Un tableau en annexe résume les différents types étudiés avec les valeurs minimales et maximales supportées

I. Introduction

Un programme durant son exécution peut avoir besoin de communiquer avec le reste du matériel. Ces échanges se résument sous le nom d'entrées-sorties, souvent abrégés E/S ou I/O (Input/Output).

Les entrées désignent les informations fournies au programme par son environnement à l'aide des périphériques d'entrées (clavier, souris, la carte réseau, etc.) et les sorties les informations fournies par le programme vers les périphériques de sorties (écran, imprimante, etc.).

Ces opérations d'entrées-sorties se feront principalement à l'aide des fonctions disponibles dans les librairies standard du langage C. La librairie C à travers la bibliothèque standard **<stdio.h>** fournie 4 fonctions utilisables pour les opérations d'entrée/sortie : **putchar()**, **printf()**, **getchar()**, **scanf()**.

II. Les sorties

Une fonction est une instruction isolée du reste du programme, qui possède un nom, et qui peut être appelée par ce nom afin d'effectuer une tâche donnée. Une fonction peut avoir besoin de paramètres dans le but de pouvoir accomplir la tâche souhaitée.

1. La fonction putchar

La fonction **putchar** permet l'affichage d'un caractère à l'écran. Elle ne manipule que des caractères, c'est-à-dire des octets dont le contenu correspond au code ASCII du caractère. Elle écrit le caractère passé en argument sur la sortie standard (l'écran).

Syntaxe :

```
putchar(<caractère>);
```

2. La fonction `printf`

La fonction **`printf`** (de l'anglais `print formatted`) disponible dans l'entête `<stdio.h>` de la librairie standard du C nous permet de faire afficher à l'écran la valeur d'une variable, un libellé, la valeur d'une expression, une chaîne de caractères, et cela, selon divers formats (associés aux différents types de variables).

Prototype :

```c
int printf(const char* format, ...);
```

Les `...` signifient que `printf` est une fonction variadique, c'est-à-dire, elle peut prendre un nombre variable de paramètres. `format` représente, comme son nom l'indique, de quoi sera faite la sortie (`Entier`, `Double`...). Son principe est le suivant : à chaque fois qu'il y a un `%`, `printf` regarde la lettre qui suit ce `%` et écrit la variable qui correspond dans les paramètres. Si c'est le `ième %`, `printf` regarde le `(ième + 1)` paramètre.

Exemples :

```c
1. #include <stdio.h>
2.
3. int main()
4. {
5.       printf("Une chaine de caractères");
6.       return 0;
7. }
```

Figure 4: Utilisation de la fonction printf avec un seul paramètre

Dans le cas où nous souhaitons utiliser plusieurs paramètres, la lettre qui suit les « **%** » dans le format correspond à un type de variable. Ces lettres sont appelées indicateurs de conversion et sont présentées dans le tableau suivant :

Table 1: Tableau résumant les indicateurs de conversion

Type	**Indicateur(s) de conversion**
`_Bool`	%d (base 10) ou `%i` (détecte la base)
`char`	%c
`string (char*)`	%s
`signed char`	%d (ou `%i`)

unsigned char	%u, %x (ou %X) ou %o
signed short	%hd
unsigned short	%hi
int	%d (base 10) ou %i (détecte la base)
unsigned int	%u ou %x (ou %X) ou %o
long	%ld ou %li
long long	%lld (ou %lli)
unsigned long	%lu, %lx (ou %lX) ou %lo
unsigned long long	%llu, %llx (ou %llX) ou %llo
float	%f, %e (ou %E) ou %g (ou %G)
double	%f, %e (ou %E) ou %g (ou %G)
long double	%Lf, %Le (ou %LE) ou %Lg (ou %LG)
Un entier octal (base 8)	%o
Un entier hexadécimal (base 16)	%x ou %X
Un pointeur (void*)	%p
Caractère %	%%

Exemples :

```c
#include <stdio.h>

int main() {
    int age = 30;

    printf("age = %d", age);

    return 0;
}
```

Figure 5: affichage d'un entier avec l'indicateur d'entier.

3. Gabarit d'affichage

Un nombre placé après le symbole % dans le format précise un gabarit d'affichage, c'est-à-dire un nombre minimal de caractères à utiliser. Si le nombre peut être affiché avec moins de

```c
#include <stdio.h>

int main()
{
	double i = 4.2;
	printf("La partie entiere du nombre %f est %d", i, (int)i);
	return 0;
}
```

Figure 6: Affichage et conversion d'un réel.

caractères qu'indiquer, la fonction `printf` le fera précéder avec le nombre d'espaces suffisant afin d'atteindre la taille indiquer. Dans le cas où le nombre de caractères indiqué est insuffisant, `printf` ignorera le gabarit et affichera le nombre avec le nombre de caractères qu'il jugera nécessaire.

Remarque : Le signe – placé immédiatement après le symbole % indique de cadrer l'affichage par la droite. Les espaces utilisés pour compléter l'affichage seront placés à droite.

Exemple :

- Le code suivant affichera à l'écran `3.141592` avec deux espaces comme gabarit.

```c
#include <stdio.h>

int main()
{
    double x = 3.141592;

    printf("%10f\n", x);

    return 0;
}
```

Figure 7: Exemple de gabarit d'affichage

- Le code suivant affichera à l'écran 2 avec trois espaces comme gabarit.

```c
#include <stdio.h>
int main()
{
  char digit = '2';

  printf("%5c\n", digit);

  return 0;
}
```

Figure 8: Exemple de gabarit d'affichage avec un caractère

4. Précision d'affichage des nombres flottants

Un nombre placé après le point décimal correspond au nombre de chiffres suivant la virgule qui seront affichés. Elle prend la forme d'un point suivi par un nombre : la quantité de chiffres qu'il y aura derrière la virgule. Cependant, notons que pour respecter la précision demandée, la valeur est arrondie.

Exemple :

Le code suivant affichera à l'écran `3.14`

```c
#include <stdio.h>

int main()
{
    double x = 3.141592;

    printf("%.2f\n", x);

    return 0;
}
```

Figure 9: Affichage d'un réel avec une précision de 2

III. Les entrées

1. La fonction `getchar`

La fonction **`getchar`** est une fonction sans paramètre qui permet de lire un caractère depuis l'entrée standard (le clavier) et le retourne. Quand un programme exécute cette fonction, le

curseur dans la fenêtre d'exécution est « en attente », l'utilisateur doit alors taper au moins un caractère du clavier. On dit qu'elle est une instruction bloquante.

Syntaxe :

```
getchar();
```

Exemple :

```
 1. #include <stdio.h>
 2.
 3. int main(void)
 4. {
 5.     char ch;
 6.
 7.     printf("Tapez un caractère: ");
 8.     ch = getchar();
 9.
10.     printf("Vous avez taper %c\n", ch);
11.
12.     return 0;
13. }
```

Figure 10 : Exemple d'utilisation de getchar

2. La fonction scanf

Grâce à la fonction **scanf** (de l'anglais **scan formatted**), il est possible de lire des informations depuis le clavier. Déclarée dans l'entête **<stdio.h>**, cette fonction peut être utilisée pour la saisie de données formatées, qu'il s'agisse de lettres, de chiffres ou de chaîne de caractères.

Syntaxe :

```
int scanf(const char * format, …);
```

Tout comme la fonction `printf`, les … signifient que c'est une fonction variadique qui peut prendre un nombre variable de paramètres. La fonction retourne le nombre de variables affectées par la saisie, et permet donc de vérifier si la saisie s'est bien passée.

2.1. Fonctionnement :

La fonction `scanf` recopie la saisie de l'utilisateur à une adresse mémoire. Cette adresse doit être spécifiée dans la fonction `scanf` (ici la saisie sera enregistrée à l'adresse de nombre, d'où le

& devant nombre). Il est possible de capturer plusieurs types de données avec la fonction `scanf`. Le type de donnée attendu est défini par le premier argument de `scanf` :

Exemple :

```c
#include <stdio.h>

int main(void)
{
    int annee;

    printf("Quelle est votre année de naissance ? ");
    scanf("%d", &annee);

    printf("Votre age est de %d an(s)", 2024 - annee);

    return 0;
}
```

Figure 10: Exemple d'utilisation de scanf.

Ici, `scanf()` attendra patiemment que l'utilisateur saisisse un nombre au clavier afin de modifier la valeur de la variable `annee`. On dit que c'est une fonction **bloquante**, car elle suspend l'exécution du programme tant que l'utilisateur n'a rien saisie.

2.2. Spécificateurs de format

Table 2: Identificateur de conversion pour scanf

Types	Identificateur(s) de conversion
`char`	%c
`signed char`	%hhd (ou %hhi)
`unsigned char`	%hhu, %hhx ou %hho
`String (chaîne de caractères)`	%s
`int`	%d ou %i
`long`	%ld ou %li
`long long`	%lld ou %lli
`unsigned long long`	%llu, %llx, ou %llo

float	%f
double	%lf
long double	%Lf

2.3. Spécificateurs de longueur

Le spécificateur de longueur est un nombre décimal positif qui permet de spécifier le nombre maximal de caractères à lire pour un champ donné. Par exemple, `%10s` signifie que jusqu'à `10` caractères seront lus et stockés dans une variable de chaîne. Les spécificateurs de largeur ne comprennent pas le terminateur `null` qui marque la fin de la chaîne. Par conséquent, la taille du tampon doit être au moins supérieure de un au spécificateur de largeur pour éviter le débordement du tampon.

Exemple :

Cet exemple permet de récupérer les 4 premiers caractères maximum du prénom au clavier. Dans le cas où le prénom dépasse 4 caractères, seuls les 4 premiers caractères sont lus et le reste des caractères lus reste dans le buffer.

```c
#include <stdio.h>
int main()
{
  char prenom[5];

  printf("Quel est votre prenom ? ");
  scanf("%4s", prenom);

  printf("\nBonjour %s\n", prenom);

  return 0;
}
```

3. Les fonctions gets et fgets

3.1. La fonction gets

La fonction `gets` permet de lire une chaine de caractères au clavier incluant les caractères espace ou tabulation. Seul le caractère `Entrée` (fin de ligne) a droit à un traitement particulier,

puisqu'il interrompt la lecture et est éliminée du tampon clavier sans être mémorisée dans la chaîne (alors que `scanf` conserve le séparateur dans le tampon clavier).

Considérons le programme suivant :

```c
#include <stdio.h>

int main()
{
    char nom[10];
    char sexe;

    printf("Quel est ton nom ? ");
    scanf("%s", &nom);

    printf("Quel est ton sexe ? (m/f) ");
    scanf("%c", &sexe);

    printf("\n\n");
    printf("Nom: %s\n", nom);
    printf("Sexe: %c\n", sexe);

    return 0;
}
```

La saisie du nom par l'utilisateur suivi de l'appui de la touche Entrée ne laissera pas la chance de saisir le sexe. Cela est dû au caractère `'\n'` resté le tampon qui se retrouve lu par la seconde instruction `scanf`. Une solution est de nettoyer ce tampon mémoire manuellement.

Le programme devient :

```c
int main()
{
    char nom[10];
    char sexe;

    printf("Quel est ton nom ? ");
    scanf("%s", &nom);

    char c;
    while(c = getchar() != '\n');

    printf("Quel est ton sexe ? (m/f) ");
    scanf("%c", &sexe);

    printf("\n\n");
    printf("Nom: %s\n", nom);
    printf("Sexe: %c\n", sexe);

    return 0;
}
```

La fonction `gets` ne souffre pas de ce problème vu qu'il arrête la lecture automatiquement au premier caractère `'\n'` rencontré.

Le programme réécris avec `gets` est :

```c
#include <stdio.h>

int main()
{
    char nom[10];
    char sexe;

    printf("Quel est ton nom ? ");
    gets(nom);

    printf("Quel est ton sexe ? (m/f) ");
    scanf("%c", &sexe);

    printf("\n\n");
    printf("Nom: %s\n", nom);
    printf("Sexe: %c\n", sexe);

    return 0;
}
```

Cependant elle nous expose à d'autre risque de sécurité comme le dépassement de tampon (`buffer overflow`) puisse qu'il n'effectue aucune vérification sur la taille des caractères lu et la taille attendue contrairement à la fonction `fgets` qui offre cette protection.

3.2. La fonction fgets

La fonction `fgets` est plus sécurisée et conseillée, car :

- Elle offre un contrôle sur le nombre de caractère à lire, ce qui permet d'éviter les dépassements de mémoire tampon, qui sont une vulnérabilité de sécurité courante dans la programmation C. Contrairement à la fonction `gets` vu précédemment, qui ne vérifient pas les limites, `fgets` garantit que l'entrée ne dépasse pas la taille allouée pour le tampon, renforçant ainsi la sécurité du programme
- Elle conserve les caractères de nouvelle ligne dans l'entrée, ce qui facilite la gestion des données textuelles formatées.

Le programme précédent avec `fgets` devient :

```c
#include <stdio.h>

int main()
{
    char nom[10];
    char sexe;

    printf("Quel est ton nom ? ");
    fgets(nom, 10, stdin);


    printf("Quel est ton sexe ? (m/f) ");
    scanf("%c", &sexe);

    printf("\n\n");
    printf("Nom: %s\n", nom);
    printf("Sexe: %c\n", sexe);

    return 0;
}
```

I. Introduction

Le langage C comme la plupart des langages modernes, fournit une panoplie d'opérateurs. Comme opérateurs, nous disposons :

- des opérateurs classiques (arithmétiques, relationnels et logiques),
- des opérateurs de manipulation de bits (bit à bit),
- des opérateurs d'affectation
- des opérateurs d'incrémentation et de décrémentation,
- des opérateurs d'adresse et de contenu (pointeurs)
- etc...

II. Les opérateurs

1. Les opérateurs arithmétiques

La construction d'expressions nécessite l'utilisation d'opérateurs arithmétiques synthétisés dans la table suivante :

Table 3: Les opérateurs arithmétiques

Signe	Signification
+	Addition
-	Soustraction
*	Multiplication
/	Division
%	Modulo
-	Nombre négatif

Les opérateurs susmentionnés, à l'exception de l'opérateur nombre négatif, sont dits binaires, car ils portent sur deux opérandes. L'opérateur nombre négatif porte sur un seul opérande, il est donc dit unaire.

Exemple :

```c
#include <stdio.h>

int main(void)
{
    int x, y;

    x = 5;
    y = 2;

    printf("%d + %d = %d\n", x, y, x + y);

    printf("%d - %d = %d\n", x, y, x - y);

    printf("%d / %d = %d\n", x, y, x / y);

    printf("%d %% %d = %d\n", x, y, x % y);

    printf("x = %d, y = %d\n", x, y);

    printf("-x = %d, -y = %d\n", -x, -y);

    return 0;
}
```

Figure 11: Exemple d'application des opérateurs arithmétiques

Remarque : Notons qu'en C le quotient de deux nombres entiers donne comme type de résultat un entier et le quotient de deux flottant fournit un nombre flottant.

2. Les opérateurs relationnels ou de comparaisons

Les opérateurs de comparaisons permettent de comparer des expressions. Notons que le langage C ne dispose pas de valeurs booléennes et utilise une convention logique pour laquelle une valeur de `0` représente faux et toute autre valeur que `0` signifie vrai.

Table 4: Opérateurs de comparaison

Opérateur	Signification
<	Inférieur à
<=	Inférieur ou égale à

>	Supérieur à
>=	Supérieur ou égale à
==	Égale à
!=	Différent de

Remarque :

- Ne pas confondre l'opérateur d'affectation simple **=** avec l'opérateur de test d'égalité **==** du tableau ci-dessus.
- Les quatre premiers opérateurs (<, <=, >, >=) possèdent la même priorité. Les deux derniers (== et !=) possèdent également la même priorité, mais leurs priorités sont inférieures à celles des précédents.
  Par exemple, dans l'expression a = b = c, l'opérateur d'affectation = a une associativité de droite à gauche, donc l'expression est équivalente à a = (b = c).
- Lorsque plusieurs opérateurs de même priorité apparaissent dans une expression, l'associativité détermine l'ordre d'évaluation. Dans L'expression a < b == c < d est interprétée comme (a < b) == (c < d).
- En cas de doute sur l'ordre d'évaluation des opérateurs, il est préférable d'ajouter des parenthèses pour clarifier l'ordre d'exécution.

Signe	Signification	Binaire ou unaire
&&	et	Binaire
||	ou	Binaire
!	non	Unaire

3. Les opérateurs logiques

Le langage C dispose de trois opérateurs logiques classiques : Le **ET** logique (noté **&&**), le **OU** logique noté (**||**) et le **NON** (noté **!**).

- L'opérateur ET logique && produit la valeur **1** si les deux opérandes ont des valeurs différentes de zéro (vrai). Si au moins un des opérandes est égal à 0, le résultat est 0. Si le premier opérande est égal à **0**, le second opérande n'est pas évalué.

- L'opérateur OU logique (`||`) effectue une opération OU inclusive sur ses opérandes. Le résultat est **`0`** si les deux opérandes ont des valeurs de **`0`**. Si au moins un des opérandes a une valeur différente de zéro, le résultat est **`1`**. Si le premier opérande d'une opération **OU** logique a une valeur différente de zéro, le second opérande n'est pas évalué.

Table 5: Fonctionnement des opérateurs logiques

Opérande 1	Opérateur	Opérateur 2	Résultat
0	&&	0	0
0		non nul	0
non nul		0	0
non nul		non nul	1
0	||	0	0
0		non nul	1
non nul		0	1
non nul		non nul	1
	!	0	1
		non nul	0

Exemple :

```c
#include <stdio.h>

int main(void)
{
    int x, y, resultat;

    x = 5;
    y = 2;

    resultat = (x > 0) && (y > 0);
    printf ("(x > 0) && (y > 0) = %d\n", resultat);

    resultat = (x >= 5) && (y >= 3);
    printf ("(x >= 5) && (y >= 3) = %d\n", resultat);

    resultat = (x >= 5) || (y >= 3);
    printf ("(x >= 5) || (y >= 3) = %d\n", resultat);

    resultat = !resultat;

    printf ("!resultat = %d\n", resultat);

    return 0;
}
```

Figure 12: Exemple d'utilisation des opérateurs logiques

4. Les opérateurs de bit à bit

Les opérateurs bit à bit s'exécutent au niveau de chaque bit (digit) qui composent un octet. Le langage C dispose des opérateurs suivants :

Opérateurs	Signification	Rôle
~	Complément à un	Inverse chaque bit d'une valeur
&	ET logique	Renvoie **1** si les deux bits correspondants sont à **1**, sinon **0**
|	OU logique	Renvoie **1** si au moins un des deux bits correspondants est à **1**, sinon **0**
^	OU exclusif	Renvoie **1** si les deux bits correspondants sont différents, sinon **0**
>>	Décalage à droite	Déplace les bits d'une valeur vers la droite d'un certain nombre de positions, en insérant des **0** à gauche
<<	Décalage à gauche	Déplace les bits d'une valeur vers la gauche d'un certain nombre de positions, en insérant des **0** à droite

Exemple :

Si `a = 00101101` et `b = 11010010`,
`~a = 11010010`, `a & b = 00000000`, `a | b = 11111111`

`a ^ b = 11111111,  a >> 2 = 00001011,  a << 3 = 01101000`

5. Les opérateurs d'affectation

5.1. L'affectation simple

L'opérateur d'affectation simple (**=**) assigne son opérande droit à son opérande gauche (on parle de **lvalue**). La valeur de l'opérande droit est convertie dans le type de l'expression d'assignation, dans le cas les deux ne sont pas de même type. La valeur de l'opérande droit remplace la valeur stockée dans l'objet désigné par l'opérande gauche.

Exemple :

```c
#include <stdio.h>

int main(void)
{
    double a = 14.25;
    int b = a;

    printf("a = %f, b = %i", a, b);
}
```

Dans cet exemple, la valeur de a est convertie en type entier et assignée à b.

5.2. L'affectation composée

Le langage C permet de combiner un opérateur d'affectation simple et un opérateur binaire (arithmétique, logique ou de décalage) à l'aide des opérateurs composée. Cela a pour effet de raccourcir les instructions et d'améliorer la lisibilité. Ils exécutent l'opération spécifiée par l'opérateur supplémentaire, puis assignent le résultat à l'opérande gauche.

Le tableau ci-dessous résume les opérateurs d'affectations composées offerts par le langage C :

Table 6: Résumé des opérateurs composées

Opérateur	Description	Exemple
+=	Additionne la valeur de droite à celle de gauche et affecte le résultat à gauche	`a += b;` équivaut à `a = a + b;`
-=	Soustrait la valeur de droite de celle de gauche et affecte le résultat à gauche	`a -= b;` équivaut à `a = a - b;`
***=**	Multiplie la valeur de droite par celle de gauche et affecte le résultat à gauche	`a *= b;` équivaut à `a = a * b;`
/=	Divise la valeur de gauche par celle de droite et affecte le résultat à gauche	`a /= b;` équivaut à `a = a / b;`
%=	Calcule le reste de la division de gauche par droite et affecte le résultat à gauche	`a %= b;` équivaut à `a = a % b;`
&=	Effectue un ET logique bit à bit entre les deux valeurs et affecte le résultat à gauche	`a &= b;` équivaut à `a = a & b;`
|	Effectue un OU logique bit à bit entre les deux valeurs et affecte le résultat à gauche	`a |= b;` équivaut à `a = a | b;`
^=	Effectue un OU exclusif bit à bit entre les deux valeurs et affecte le résultat à gauche	`a ^= b;` équivaut à `a = a ^ b;`
<<=	Décale les bits de gauche vers la gauche d'un nombre de positions égal à la valeur de droite et affecte le résultat à gauche	`a <<= b;` équivaut à `a = a << b;`
>>=	Décale les bits de gauche vers la droite d'un nombre de positions égal à la valeur de droite et affecte le résultat à gauche	`a >>= b;` équivaut à `a = a >> b;`

5.3. Les opérateurs d'incrémentation et de décrémentation

Les opérateurs d'incrémentation et de décrémentation sont des opérateurs qui permettent de modifier la valeur d'une variable en ajoutant ou en soustrayant 1. Ils sont souvent utilisés dans les boucles comme avancement afin de contrôler le nombre d'itérations.

Le langage C dispose de deux formes d'opérateurs d'incrémentation et de décrémentation : la forme préfixée et la forme suffixée :

a. La forme préfixée

La forme préfixée modifie la valeur de la variable avant de l'utiliser dans une expression.

Exemple :

Supposons une variable **a** de type entier déclarée avec une valeur de **10**

- L'instruction **++a**, est une incrémentation préfixée, qui ajoute **1** à a et renvoie la nouvelle valeur de **a**, c'est-à-dire **11**.
- L'instruction **--a**, est une décrémentation préfixée, qui soustrait **1** à a et renvoie la nouvelle valeur de a, c'est-à-dire **9**.

b. La forme suffixée

La forme suffixée modifie la valeur de la variable après l'avoir utilisée dans une expression.

Exemple :

Supposons une variable **a** de type entier déclarée avec une valeur de **10**

- L'instruction **a++** c'est une incrémentation suffixée, qui renvoie la valeur actuelle de a, c'est-à-dire **10**, puis ajoute **1** à **a**.
- L'instruction **a--** : c'est une décrémentation suffixée, qui renvoie la valeur actuelle de a, c'est-à-dire **10**, puis soustrait **1** à **a**.

NB : Les opérateurs d'incrémentation et de décrémentation sont des opérateurs unaires.

Application : Compléter le tableau suivant

Avant instruction		**Instruction**	**Après instruction**	
Valeurs			**Valeurs**	
x	y		x	y
1	0	`y = x++`		
2	3	`y = x--`		
7	10	`y = ++x + y--`		
8	7	`y = --x`		

5.4. L'opérateur de casting

L'opérateur de casting en C est un opérateur qui permet au programmeur de forcer la conversion d'une expression d'un type de données à un autre type de données. Il est utilisé dans des situations où la conversion implicite n'est pas possible ou souhaitable.

Il peut être aussi utilisé pour modifier le type d'une variable, d'une constante, d'un résultat d'opération, d'un argument de fonction, ou d'un retour de fonction.

La syntaxe générale de l'opérateur de casting en C est la suivante :

Syntaxe :

```
(<nom-du-type>) <expression>;
```

Le **`nom-du-type`** est le nom du type de données standard du langage C, et **`expression`** est l'expression à convertir.

Exemple :

Si l'on veut convertir un entier en un réel, nous pouvons écrire : `(float) x`

L'opérateur de casting en C est utile pour éviter la perte de données, la troncature, ou les erreurs de précision lors de la manipulation de différents types de données. Il est aussi nécessaire pour utiliser certaines fonctions de la bibliothèque standard qui attendent des arguments d'un type spécifique.

I. Introduction

Les structures conditionnelles sont des instructions qui permettent de contrôler le flux d'exécution d'un programme en fonction de la réalisation de différentes conditions. Elles sont utiles pour réaliser des choix, des alternatives ou des répétitions selon des critères logiques. Le langage C dispose de plusieurs types de structures conditionnelles dont : **`if`**, **`if`**... **`Else`**, **`if`**... **`Elseif`**... **`Else`** et **`switch`**.

II. Les structures if

1. La structure `if` simple

Elle exécute un bloc d'instruction si une condition est vraie ou de le passer si ce n'est pas le cas.

Syntaxe :

```
1. if (<expression>) {
2.     <bloc d'instructions>
3. }
```

NB :

- La partie **`<expression>`** peut désigner : une variable d'un type numérique, une expression fournissant un résultat numérique.
- La valeur de la condition utilise la convention logique du langage C : **`0`** est faux, autre chose que **`0`** est vrai.
- Si la condition n'est pas vérifiée, le bloc d'instructions est passé et le programme continue immédiatement à la suite du bloc d'instructions délimité par l'instruction **`if`**.
- Dans le cas où le bloc d'instruction ne contient qu'une instruction à réaliser, les accolades peuvent être omises.

Exemple :

```c
#include <stdio.h>

int main() {
    int a = 10;
    int b = 20;

    if (a < b) {
        printf("%d est plus petit que %d\n", a, b);
    }

    return 0;
}
```

2. La structure `if...else`

Elle exécute un bloc d'instructions si une condition est vraie, et un autre bloc sinon.

Syntaxe :

```
if (<expression>) {
    <bloc d'instruction 1>
} else {
    <bloc d'instruction 2>
}
```

NB :

- Si l'***<expression>*** fournit une valeur différente de **0**, alors le **<bloc d'instructions 1>** est exécuté.
- Si l'**<expression>** fournit la valeur **0**, alors le **<bloc d'instructions 2>** est exécuté.

Exemple :

```c
#include <stdio.h>

int main() {
    int age;

    printf("Quelle est votre age? ");
    scanf("%d", &age);

    if(age >= 18) {
       printf("Selon la loi togolaise, vous êtes majeur");
    } else {
       printf("Selon la loi togolaise, vous êtes mineur");
    }

    return 0;
}
```

3. L'instruction `if…elseif…else`

Elle exécute un bloc d'instructions parmi plusieurs selon la première condition vraie rencontrée à travers l'imbrication de **`if`**.

Syntaxe :

```
if(<expression 1>) {
    <bloc d'instruction 1>
} else if(<expression 2>) {
    <bloc d'instruction 2>
} else {
    <bloc d'instruction 3>
}
```

NB :

- Nous pouvons avoir autant de bloc **`else if`** que nécessaire

Application :

Écrire un programme qui prend en entrée le prix hors taxes d'un produit, la TVA, le jour de la semaine (en utilisant des valeurs numériques, par exemple, **1** pour lundi, **2** pour mardi, etc.) et calcule le prix **TTC** en fonction de la remise applicable.

- Si le prix hors taxes est inférieur à **`50 000`** francs, la remise est de **5** %.

- Si le prix hors taxes est compris entre **50 000** et **100 000** francs (inclus), la remise est de **10** %.
- Si le prix hors taxes est supérieur à **100 000** francs, la remise est de **15** %.
- Si le jour de la semaine est un samedi ou un dimanche, une remise supplémentaire de **2** % est appliquée.
- Le programme doit afficher le prix initial, la remise appliquée et le prix final **TTC**.

III. L'instruction `switch`

L'instruction **`switch`** permet de comparer la valeur d'une expression entière par rapport à une liste de constantes entières. Elle représente une simplification des instructions **`if…elseif…else`**.

Syntaxe :

```
switch(<expression>)
{
    case <constante 1>:
      <instruction 1>
      break;
    case <constante 2>:
      <instruction 2>
      break;
    …
    default:
      <instruction 1>
}
```

NB :

- Une **<constante n>** doit avoir une valeur de type entier ou caractère constante non ambiguë connue au moment de la compilation ;
- L'instruction **`switch`** peut inclure un nombre quelconque d'instances case. Toutefois, deux valeurs **<constante n>** ne peuvent pas être identiques dans la même instruction **`switch`** ;
- L'exécution du corps de l'instruction **`switch`** commence à la première instruction **`case`** correspondante. L'exécution se poursuit jusqu'à la fin du corps, ou jusqu'à ce qu'une instruction **`break`** transfère le contrôle hors du corps.

- L'instruction **default** est exécutée si aucune valeur **case** n'est égale à la valeur de **<expression>** et elle peut apparaître n'importe où dans le corps de l'instruction **switch**

```c
#include <stdio.h>

int main()
{
    int jour;

    printf("Quel est le numéro du jour? ");
    scanf("%d", &jour);

    switch(jour) {
        case 1:
            printf("C'est Lundi");
            break;
        case 2:
            printf("C'est Mardi");
            break;
        case 3:
            printf("C'est Mercredi");
            break;
        case 4:
            printf("C'est Jeudi");
            break;
        case 5:
            printf("C'est Vendredi");
            break;
        case 6:
            printf("C'est Samedi");
            break;
        case 7:
            printf("C'est Dimanche");
            break;

        default:
          printf("Ce numéro de jour est incorrect");
    }

    return 0;
}
```

Application : Modifiez ce programme de sorte qu'il affiche « `C'est un jour de travail` » pour les jours de travail et « `C'est le weekend, on s'éclate` » pour les jours du weekend.

IV. L'opérateur conditionnel

L'opérateur conditionnel ou ternaire est un opérateur qui permet d'évaluer une expression selon une condition. Il est nommé opérateur ternaire parce qu'il est le seul opérateur du langage C à requérir trois opérandes : Un premier opérande qui représente la condition et les deux autres opérandes représentent l'expression.

Syntaxe :

```
<condition> ? <expression si vrai> : <expression si faux>
```

Exemple :

```c
#include <stdio.h>
int main()
{
  int a = 14;
  int b = 20;
  int max;

  max = a > b ? a : b;

  printf("Max = %d", max);

  return 0;
}
```

I. Introduction

Les structures répétitives en langage C sont des instructions qui nous permettent de répéter l'exécution d'un bloc de code selon une condition ou un nombre de fois. Le langage C dispose de trois structures : Les boucles **for**, **while** et **do... While**.

II. La boucle for

La boucle **for** est la boucle la plus courante en C. Elle est l'équivalent de la structure **Pour** étudiée au cours d'initiation à l'algorithmique et est utilisée lorsque le nombre de répétitions est connu à l'avance ou comme boucle de comptage.

La syntaxe de la boucle **for** est la suivante :

Syntaxe :

```
1. for (<initialisation>; <condition>; <incrémentation>) {
2.     <bloc d'instructions>
3. }
```

NB :

- La partie initialisation est exécutée une seule fois, avant le début de la boucle. Elle est utilisée pour initialiser les variables utilisées dans la boucle.
- La partie condition est évaluée avant chaque passage de la boucle. Si la condition est vraie, c'est-à-dire une valeur différente de **0**, le bloc d'instructions est exécuté. Sinon, la boucle est terminée.
- La partie incrémentation est exécutée après chaque passage de la boucle. Elle est utilisée pour incrémenter ou décrémenter les variables utilisées dans la boucle.

Exemple :

```c
#include <stdio.h>
int main(void)
{
  for (int i = 1 ; i <= 100 ; i++) {
    printf("%d\n", i);
  }

  return 0;
}
```

Figure 13: Programme qui affiche les nombres de 1 à

III. La boucle While

La structure **`while`** est l'équivalent en langage C de la structure **`TantQue`** vue au cours d'initiation à l'algorithmique. Elle est utilisée lorsque le nombre de répétitions n'est pas connu à l'avance. Sa syntaxe est la suivante :

Syntaxe :

```
While(<expression>)
<bloc d'instructions>
```

NB :

- La partie **`<expression>`** peut désigner : une variable d'un type numérique, une expression dont l'évaluation fournie un résultat numérique ;
- La partie **`<bloc d'instructions>`** peut être une seule ligne d'instruction terminée par un point-virgule (Dans ce cas les parenthèses ne sont pas nécessaires) ou un bloc d'instructions compris entre accolades ;
- Tant que l'**`<expression>`** fournit une valeur différente de zéro, le **`<bloc d'instructions>`** est exécuté et la répétition s'arrête avec une valeur de **`0`**.

Exemple : Que fait le programme ci-dessous ?

```c
#include <stdio.h>
int main(void)
{
  int i;

  i = 1;
  while (i <= 100) {
    if(i % 2 == 0) {
      printf("%d\n", i);
    }

    i++;
  }

  return 0;
}
```

IV. La boucle do…while

La boucle do…while est similaire à la boucle while, mais la condition est placée à la fin de la boucle. Cette position de la condition implique qu'elle est évaluée après chaque passage de la boucle. Le <bloc d'instructions> est donc toujours exécuté au moins une fois, même si la condition est fausse.

La syntaxe de la boucle do…while est la suivante :

```c
do {
    <bloc d'instructions>
} While(<expression>);
```

Exemple :

```c
#include <stdio.h>
int main(void)
{
  int age;

  do {
    printf("Quel est votre age ? ");
    scanf("%d", &age);

    if (age < 18) {
      printf("Les mineurs n'entrent pas ici\n");
    }

  } while(age <= 18);

  printf("Bienvenu..\n");

  return 0;
}
```

V. Choix d'une boucle

Le choix de la structure répétitive à utiliser dépend du contexte. Dans le cas où le nombre de répétitions est connu à l'avance, la boucle **for** est généralement la meilleure option. Si le nombre de répétitions n'est pas connu à l'avance, la boucle **while** ou la boucle **do…while** peuvent être utilisées. Cependant, on préfèrera la boucle **do…while** dans les situations où une première exécution ou un contrôle de saisie est nécessaire.

Applications : Affichez la table suivante des produits pour ***n***. La valeur de ***n*** sera demandée à l'utilisateur. Cet exemple présente le cas où ***n*** vaut ***10***.

```
X*Y  I   0   1   2   3   4   5   6   7   8   9  10
-------------------------------------------------
 0   I   0   0   0   0   0   0   0   0   0   0   0
 1   I   0   1   2   3   4   5   6   7   8   9  10
 2   I   0   2   4   6   8  10  12  14  16  18  20
 3   I   0   3   6   9  12  15  18  21  24  27  30
 4   I   0   4   8  12  16  20  24  28  32  36  40
 5   I   0   5  10  15  20  25  30  35  40  45  50
 6   I   0   6  12  18  24  30  36  42  48  54  60
 7   I   0   7  14  21  28  35  42  49  56  63  70
 8   I   0   8  16  24  32  40  48  56  64  72  80
 9   I   0   9  18  27  36  45  54  63  72  81  90
10   I   0  10  20  30  40  50  60  70  80  90 100
```

I. Introduction

Les variables simples étudiées jusqu'ici ne permettent de stocker qu'une seule valeur donnée à la fois. Ce comportement ne nous permet pas de faire face aux situations où nous avons plusieurs valeurs distinctes à gérer. Heureusement, le langage C dispose de structures de données nous permettant de stocker l'ensemble de ces données dans une « variable commune ». Ainsi, pour accéder à ces valeurs, il suffit de parcourir la variable de type complexe composée de « variables » de type simple.

Le langage C dispose de deux types de tableaux : les tableaux statiques, dont la taille est connue à la compilation, et les tableaux dynamiques, manipulés à l'aide des pointeurs, ils ont leur taille qui est connue à l'exécution.

Dans ce chapitre nous nous intéresseront aux tableaux statiques.

II. Notion de tableau

Un tableau est une variable composée pouvant stocker des données de même type. Ces données sont stockées de manière contiguë en mémoire (les unes à la suite des autres) et possèdent tous la même taille. Une valeur du tableau porte le nom d'élément.

Un tableau est donc une suite de cases (espace mémoire possédant une adresse) de même taille. La taille de chacune des cases est conditionnée par le type de donnée que le tableau contient.

Les éléments du tableau peuvent être :

- Des données de type simple : **`int`**, **`char`**, **`float`**, **`long`**, **`double`**...

  Dans ce cas, la taille de chaque case est donnée par le nombre d'octets sur lequel est codé le type considéré.

- Des pointeurs (objets contenant une adresse mémoire
- Des tableaux
- Des structures

NB :

- Lorsque le tableau est composé de données de type simple, on parle de tableau unidimensionnel (ou vecteur) ;
- Lorsque celui-ci contient lui-même d'autres tableaux on parle alors de tableaux multidimensionnels (aussi matrice ou table).

III. Les tableaux unidimensionnels

1. Déclaration

Syntaxe :

```
<type> <identificateur> [<taille>];
```

NB :

- *<type>* définit le type d'élément que contient le tableau, c'est-à-dire qu'il définit la taille d'une case du tableau en mémoire.
- `<identificateur>` est le nom que l'on décide de donner au tableau, le nom du tableau suit les mêmes règles qu'un nom de variable.
- `<taille>` indique le nombre d'éléments est un nombre entier qui détermine le nombre de cases que le tableau doit comporter.

Exemple :

```
int notes[8];
```

Cet exemple déclare un tableau nommé notes qui peut contenir 8 éléments de type `int` (8 entiers).

2. Initialisation du tableau

a. Initialisation avec des valeurs

Il est possible d'initialiser le tableau lors de sa déclaration.

Exemple :

```
int notes[8] = {14, 12, 5, 4, 8, 17, 16, 8};
```

b. Initialisation sans précision de taille

Dans le cas où aucune taille n'est précisée, le compilateur calculera automatiquement la taille nécessaire pour contenir les éléments spécifiés.

Exemple :

```
int notes[] = {14, 12, 5, 4, 8, 17, 16, 8};
```

c. Initialisation par défaut

Dans le cas où nous déclarons un tableau de taille fixe, mais qu'on l'initialise avec moins d'éléments qu'il peut contenir, les éléments restants seront mis à zéro. On utilise cette astuce pour initialiser rapidement un tableau à zéro.

Exemple :

```
int notes[] = {0};
```

3. Accès à un élément

En langage C, les éléments d'un tableau sont présents à une position particulière dans le tableau. Cette position est nommée *indice* et est relatif au début du tableau. Les éléments d'un tableau sont indexés de **0** jusqu'à **`taille - 1`**, et le i^e élément peut être accédé de la manière suivante :

Syntaxe d'accès à un élément du tableau :

```
<identificateur>[<indice>]; ou <indice>[<identificateur>];
```

NB :

- Même si les deux syntaxes sont équivalentes, la seconde est peu utilisée et est moins intuitive que la première. Nous adopterons donc la première lors de ce cours.
- Chaque élément d'un tableau à un indice donné peut être utilisé comme une variable à part entière. Ils sont donc acceptés dans toutes les opérations où une variable est habituellement attendue.

Exemple : L'instruction ci-dessus nous permet d'accéder à la deuxième case du tableau notes.

```
notes[2]
```

4. Parcours à l'aide d'une boucle

À l'aide des boucles étudiées précédemment, nous pouvons parcours l'ensemble des éléments d'un tableau.

Exemple :

```c
#include <stdio.h>

#define NB_NOTE 8

int main()
{
    int notes[NB_NOTE];

    for(int i = 0; i < NB_NOTE; i++) {
        notes[i] = 10 + i;
    }

    for(int i = 0; i < NB_NOTE; i++) {
        printf("Case %d, note = %d\n", i, notes[i]);
    }

    return 0;
}
```

Application : Implémentez les algorithmes de recherche sur les tableaux étudiés au cours d'initiation à l'algorithmique en Langage C.

IV. Les tableaux multidimensionnels

Les tableaux à plusieurs dimensions, contrairement aux tableaux à une dimension, contiennent d'autres tableaux.

1. Déclaration

Syntaxe de déclaration :

```
<type> <identificateur> [<entier 1>][<entier 2>]…[<entier n>];
```

NB :

- Chaque élément entre crochets désigne le nombre d'éléments dans chaque dimension
- Le nombre de dimensions n'est pas limité

Exemple :

```
int notes[11][5];
```

Cet exemple permet de déclarer un tableau à deux dimensions pouvant contenir 55 notes.

2. Initialisation du tableau

Comme vu pour les tableaux à une dimension, les tableaux à plusieurs dimensions peuvent êtres initialisés lors de leur déclaration.

Exemple :

```
int matrice[2][3] = { { 1 , 2 , 3 } , { 4 , 5 , 6 } };
```

3. Accès aux éléments

L'accès aux éléments se fait en indiquant les valeurs associées aux différents indices. Dans le tableau précédemment déclaré, l'accès à la 5e note de l'étudiant n°7 s'écrira :

Syntaxe :

```
int notes[6][4];
```

4. Parcours à l'aide d'une boucle

L'accès aux éléments d'un tableau multidimensionnels se fait à l'aide d'un nombre de boucles correspondant aux dimensions du tableau.

```
 1. #include <stdio.h>
 2.
 3. #define NB_ETUDIANTS 11
 4. #define NB_MATIERES 5
 5.
 6. int main()
 7. {
 8.     float notes[NB_ETUDIANTS][NB_MATIERES] = {0};
 9.
10.     for(int i = 0; i < NB_ETUDIANTS; i++) {
11.         printf("Etudiant n° %d\n", i+1);
12.         for(int j = 0; j < NB_MATIERES; j++) {
13.             printf("Entez la note n°%d: ", j+1);
14.             scanf("%f", notes[i][j]);
15.         }
16.     }
17.
18.     return 0;
19. }
```

Considérant le tableau `notes` déclarés plus haut, nous pouvons accéder aux valeurs de la façon suivante :

Application :

Écrire un programme C qui permet de :

- Créer un tableau contenant les notes de 10 étudiants dans 5 matières
- Faire la saisie des notes de chaque étudiant
- Calculer et afficher la moyenne de chaque étudiant
- Calculer la moyenne générale de la classe
- D'afficher la moyenne la plus haute et la moyenne la plus basse de la classe
- D'afficher le nombre d'étudiants ayants une moyenne supérieure à 10

I. Notion de chaine de caractères

Le langage C contrairement à certains langages comme Java, C++, C#, PHP, ne dispose pas d'un véritable type chaine de caractères. Les chaînes de caractères en C sont des tableaux de caractères terminés par le caractère null ('\0').

II. Déclaration et initialisation des chaines de caractères

1. Syntaxe

La déclaration des chaines de caractères se fait comme suit :

```
char <identificateur>[taille];
```

NB : Le caractère de fin de chaine noté '\0' doit être présent dans la dernière case du tableau.

2. Initialisation d'une chaine de caractères

2.1. Initialisation du tableau caractère par caractères

L'initialisation caractère par caractères permet d'initialiser le tableau en précisant les différents caractères qui la composeront. La taille de la chaine sera devinée par le compilateur.

Exemple :

```
char texte[] = {'S','a','l','u','t','\0'};
```

2.2. Initialisation du tableau avec une chaine littérale

Nous pouvons initialiser le tableau avec une chaine de caractères. Avec cette forme d'initialisation, le compilateur ajoutera automatiquement le caractère de fin de chaine.

```
char texte[] = "Salut!";
```

3. Initialisation du tableau par lecture au clavier

Les fonctions **scanf** et **fgets** peuvent être utilisé pour la lecture d'une chaine au clavier.

3.1. Avec la fonction scanf

Afin d'éviter un débordement de tampon, il est nécessaire d'indiquer à la fonction **scanf** le nombre de caractères à lire.

```c
#include <stdio.h>

int main()
{
    char nom[20];

    printf("Quel est ton nom ? ");
    scanf("%20s", nom);

    printf("Bonjour %s !", nom);

    return 0;
}
```

3.2. Avec la fonction fgets

La fonction **fgets** attend 3 paramètres : le nom du tableau de caractères, le nombre de caractères à lire, et le périphérique sur lequel lire les caractères. Par défaut, nous passerons comme troisième paramètre **stdin** pour représenter l'entrée standard.

```c
#include <stdio.h>

int main()
{
    char nom[20];

    printf("Quel est ton nom ? ");
    fgets(nom, 20, stdin);

    printf("Bonjour %s !", nom);

    return 0;
}
```

La fonction **`fgets`** est conseillé non seulement à cause de la sécurité qu'il offre, mais aussi parce qu'il est capable de lire une ligne entière contenant des espaces.

III. Quelques fonctions de la librairie standard

Le langage C ne disposant pas de type chaine de caractères, il dispose néanmoins une librairie standard qui offre quelques fonctions usuelles sur les chaines de caractères.

Afin de tirer parti de ces fonctions, nous devons inclure l'entête `string.h`. Cette bibliothèque fournit un ensemble de fonctions qui permettent de manipuler des chaînes de caractères. En voici la liste : `strcat`, `strchr`, `strcmp`, `strcpy`, `strcspn`, `strlen`, `strncat`, `strncmp`, `strncpy`, `strpbrk`, `strrchr`, `strspn`, `strtok`.

Un tableau en annexe résume l'utilité de ces fonctions et leur fonctionnement.

Application :

1. Écrire un programme C qui lit deux chaînes de caractères et les affiche dans l'ordre alphabétique, en utilisant les deux méthodes suivantes :
    a. En utilisant la fonction `strcmp`.
    b. Sans utiliser la fonction `strcmp`.

    Par exemple, si on donne en entrée les deux chaînes suivantes : `acb` et `abcd`, le programme doit afficher la chaîne `abcd` puis `acb`.
2. Écrire un programme qui lit une chaîne de caractères et vérifie si elle est palindrome ou non. Le palindrome, du grec *palindromos*, « *qui revient sur ses pas* », est un mot ou un groupe de mots qui peut être lu indifféremment de gauche à droite ou de droite à gauche, comme par exemple les mots « kayak », « radar », « coloc », « rêver », « ressasser ».

I. La notion de fonction

Une fonction est un sous-programme qui constitue une partie indépendante du programme principal chargé d'effectuer une tâche bien précise. Elle peut être définie et appelé un nombre de fois illimité à l'intérieur d'un autre sous-programme ou à l'intérieur du programme principal. La notion de fonction en programmation partage des similarités avec celui des mathématiques.

II. Définition et appel d'une fonction

1. Déclaration d'une fonction :

Syntaxe :

```
<type de retour> nomDeLaFonction(<type> <paramètre 1>, <type> <paramètre 2>, …)
{
    <corps de la fonction>
}
```

NB :

- `<type de retour>` représente le type de Valeur que la fonction est censée retourner. Le type de retour peut être de la liste des types étudiés au Chapitre 3. Dans le cas où la fonction ne retourne pas de résultat, nous mettrons **void**.
- **`nomDeLaFonction`** représente le nom attribué à la fonction. Le nom de la fonction suit les mêmes règles que ceux des variables et doit être utilisé à chaque fois que l'on souhaite faire appel à la fonction ;
- Dans le cas où la fonction ne prend pas de paramètres, nous laisserons les parenthèses vides ou en mettant le mot clé **void** entre parenthèses.

2. Prototype d'une fonction

Un prototype de fonction est une déclaration qui donne des informations sur une fonction qui sera définie plus tard dans le programme. Comme information, il est indiqué au compilateur : le nom de la fonction, son type de retour et les types de ses paramètres.

```
<type de retour> nomDeLaFonction(<type> <paramètre 1>, <type> <paramètre 2>, …);
```

Syntaxe de déclaration de prototype :

NB :

- Le corps de la fonction n'est pas précisé dans le prototype ;
- Les … dans un prototype indiquent que la fonction accepte un nombre variable de paramètres. Dans ce cas, il ne faut pas oublier d'inclure l'entête `stdarg.h` ;
- Le prototype d'une fonction doit être placé avant son appel ou sa définition dans le programme.

3. Appel d'une fonction

Une fois une fonction définie, elle peut être appelée dans une autre fonction ou à l'intérieur du programme principal. La syntaxe d'appel d'une fonction est la suivante :

```
nomDeLaFonction(<argument 1>, <argument 2>, …);
```

NB :

- Le nombre d'arguments fournis lors de l'appel d'une fonction doit être le même que le nombre de paramètres annoncés lors de la déclaration.

4. Exemple de fonction

Ce programme ci-dessous permet de faire la somme de trois nombres saisis au clavier. Notons la définition de la fonction somme.

- La ligne 3 définie le prototype de la fonction **`somme`**. Elle annonce une fonction nommée somme qui prend en paramètre 3 arguments `(a, b, c)` de type entier. Notez que le nom des paramètres dans le prototype est facultatif.
- Les lignes 17, 18 et 19 définissent le corps de la fonction **`somme`**. L'instruction **`return`** suivi par l'expression de calcul de la somme permet de renvoyer le résultat de l'opération au programme appelant : Ici la fonction **`main`**.
- La ligne 12 permet de faire appel à la fonction **`somme`**. Notons les trois arguments passés en paramètre.

```c
#include <stdio.h>

int somme(int, int, int);

int main()
{
    int a, b, c;

    printf("Entrez a, b, c: ");
    scanf("%d %d %d", &a, &b, &c);

    printf("a + b + c = %d", somme(a, b, c));

    return 0;
}

int somme(int a, int b, int c) {
    return a + b + c;
}
```

III. Arguments et durée de vie d'une variable

1. Les arguments d'une fonction

Les arguments d'une fonction sont les valeurs passées à une fonction lors de son appel. Ils permettent à la fonction de recevoir des données en entrée et de les traiter selon son code. Les arguments sont placés entre parenthèses après le nom de la fonction, séparés par des virgules.

Un argument peut être : une constante, une variable, une expression, une autre fonction.

a. Passage par valeurs

Par défaut, les arguments sont transmis aux paramètres par valeur, ce qui signifie qu'une copie de la valeur de l'argument est faite dans le paramètre. Ainsi, toute modification du paramètre dans la fonction n'affecte pas la valeur de l'argument dans le programme appelant.

b. Passage par adresse

Pour qu'une fonction puisse modifier la valeur des arguments passés en paramètres, l'adresse des arguments doit être passé en lieu et place des valeurs. Ce type de passage se fait à l'aide des pointeurs.

2. Durée de vie des variables

La durée de vie d'une variable est le temps pendant lequel elle existe et conserve sa valeur en mémoire. Elle dépend de la classe de stockage de la variable, qui peut être statique, automatique ou dynamique.

- **Les variables statiques :** Elles sont déclarées à l'aide du mot clé `static` et sont créées au début du programme et détruites à la fin. Elles ont donc une durée de vie égale à celle du programme.
- **Les variables automatiques :** Elles sont créées à l'entrée d'un bloc de code et détruites à la sortie. Elles ont alors une durée de vie limitée à la portée du bloc.
- **Les variables dynamiques :** Elles sont créées et détruites explicitement par le programmeur, à l'aide des fonctions `malloc` et `free`. Elles sont contrôlées à l'aide des pointeurs et ont une durée de vie contrôlée par le programmeur.

Toutes les variables n'ayant pas les mêmes portées, nous distinguons les variables globales et les variables locales.

a. Variables globales

Une variable globale est une variable déclarée au début du programme en dehors de toute fonction. Elle peut être utilisée par n'importe quelle fonction du programme, sans avoir besoin de la passer en paramètre. Elle conserve sa valeur tout au long de l'exécution du programme.

b. Variables locales

Une variable locale est une variable déclarée à l'intérieur d'une fonction ou d'un bloc de code. Elle ne peut être utilisée que dans la fonction ou le bloc dans lequel elle est déclarée. Elle est créée lors de l'appel de la fonction et détruite à la fin de l'exécution de cette fonction. Sa valeur n'est pas conservée entre deux appels.

3. Exemple de programme

Le programme ci-dessous définit une fonction `echange` qui échange les valeurs des arguments passés en paramètre. Cependant, on remarque que les valeurs des arguments ne sont échangées qu'à l'intérieur de la fonction. Ceci est du fait que les arguments sont passés par valeurs. Les modifications dont ils font objet ne sont pas reflétés dans le programme principal.

```
1. #include <stdio.h>
2.
3. void echange (int, int);
4.
5. void echange (int a, int b)
6. {
7.   printf ("Debut de l'echange, a = %d; b = %d\n", a, b);
8.   int c = a;
9.   a = b;
10.  b = c;
11.  printf ("Fin de l'echange a = %d; b = %d\n", a, b);
12. }
13.
14. int main ()
15. {
16.   int a = 7;
17.   int b = 5;
18.
19.   printf ("Avant appel de la fonction echange, a = %d; b = %d\n", a, b);
20.   echange (a, b);
21.   printf ("Apres appel de la fonction echange, a = %d; b = %d\n", a, b);
22.
23.   return 0;
24. }
```

IV. Retour d'une fonction

L'exécution d'une fonction se termine soit lorsque la fin du bloc délimitant la fonction est atteinte, soit lorsque le mot clef `return` est rencontré. Le retour d'une fonction se fait à l'aide du mot-clé `return`, qui a deux effets :

- Il termine l'exécution de la fonction et retourne le contrôle à la fonction appelante
- Il évalue l'expression qui suit le mot-clé `return` et la convertit dans le type de retour de la fonction.

Le mot-clé `return` peut être suivi : d'une valeur, d'une constante, d'une expression ou d'un appel d'une autre fonction.

V. La notion de récursivité

Une fonction récursive est une fonction qui s'appelle elle-même directement ou indirectement.

- **La récursivité directe :** Une fonction est à récursivité directe si elle comporte dans ses instructions un appel vers elle-même.
- **La récursivité indirecte ou croisée :** Cette récursivité permet à une fonction de faire appel à une autre fonction qui, à son tour fait appel à la fonction originale.

1. Structure des fonctions récursives

Pour écrire une fonction récursive C, il faut respecter deux principes :

- **Il faut définir un cas de base**, qui est le scénario le plus simple qui ne nécessite pas de récursion. C'est une condition d'arrêt qui empêche la fonction de s'appeler indéfiniment. Sans un cas de base approprié, une fonction récursive C peut entraîner une récursion infinie.
- **Il faut définir un cas récursif**, dans lequel la fonction s'appelle elle-même avec des arguments modifiés. C'est l'essence de la récursion : résoudre un problème plus grand en le réduisant à des instances plus petites du même problème. Le cas récursif doit se rapprocher du cas de base à chaque itération.

2. Exemple avec la fonction factorielle

La fonction factorielle récursive est une fonction qui calcule la factorielle d'un nombre en utilisant la récursion, c'est-à-dire qu'elle s'appelle elle-même avec un argument plus petit. La factorielle d'un nombre `n`, notée `n!`, est le produit de tous les entiers positifs inférieurs ou égaux à n. Par exemple, `5! = 5 × 4 × 3 × 2 × 1 = 120`.

- Le cas de base de la fonction factorielle est `n = 0`, car `0! = 1` par convention
- Le cas récursif est `n > 0`, car `n! = n × (n - 1)!`. Ainsi, on peut calculer la factorielle de n en appelant la fonction factorielle avec `n - 1`, et en multipliant le résultat par n.

Le programme suivant est une implémentation de la factorielle

```c
#include <stdio.h>

unsigned long long factorielle(int);

int main()
{
    int a;

    printf("Entrez un nombre: ");
    scanf("%d", &a);

    printf("%d! = %llu", a, factorielle(a));

    return 0;
}

unsigned long long factorielle(int n) {
    if (n == 0)
        return 1;
    else
        return n * factorielle(n - 1);
}
```

Application :

1. Écrire une fonction, nommée `f2`, qui affiche "bonjour" un nombre de fois égal à la valeur reçue en argument (`int`) et qui ne renvoie aucune valeur.
2. Écrire une fonction nommée `CelsiusToFahrenheit` qui permet de convertir une température de Celsius en Fahrenheit avec la formule `°F = °C × (9/5) + 32`.
3. Écrire une fonction nommée `puissance` qui prend deux paramètres **x** et **y** et qui permet de calculer d'élever **x** à la puissance **y**.
4. Écrire une fonction récursive calculant la valeur de la « fonction d'Ackermann » A définie par :

$$A(m,n) = \begin{cases} n+1 & \text{si } m = 0 \\ A(m-1,1) & \text{si } m > 0 \text{ et } n = 0 \\ A(m-1,A(m,n-1)) & \text{si } m > 0 \text{ et } n > 0. \end{cases}$$

I. Notions de structures

Une structure est un type de donnée qui permet de regrouper plusieurs objets pouvant être de types différents sous un même nom. Le mot-clé **`struct`** est utilisé pour définir une structure en C. Chaque variable dans la structure est appelée un membre de la structure. Contrairement à un tableau, une structure peut contenir des types de données différents (**`int`**, **`float`**, **`char`**, etc.).

II. Définition d'une structure

1. Syntaxe de déclaration d'une structure

```
1. struct <nomDeLaStructure>
2. {
3.     <Objet(s) composant(s) la structure>
4. };
```

NB :

- Le nom donné à la structure lors de sa déclaration doit suivre les mêmes règles que celui des identificateurs ;
- La composition de la structure (sa définition) est décrite à l'aide d'une suite de déclarations de variables. Ces différentes déclarations constituent les membres ou champs de la structure ;
- Notez la présence d'un point-virgule obligatoire à la fin de la définition de la structure.

2. Déclaration et initialisation d'une variable de type structure

Une fois la structure déclarée, nous pouvons déclarer une variable du type de cette structure en suivant la syntaxe suivante :

```
struct <nomDeLaStructure> <identificateur>;
```

Exemple :

```c
#include <stdio.h>

int main()
{
    struct etudiant {
        char nom[255];
        char prenom[255];
        char sexe;
        int age;
    };

    struct etudiant etudiant1;

    return 0;
}
```

2.1. Initialisation d'une structure

Comme n'importe quelle variable, nous pouvons initialiser les valeurs des différents membres de la structure. L'initialisation s'effectue à l'aide d'une liste fournissant une valeur pour un ou plusieurs membres de la structure.

Nous disposons de plusieurs formes d'initialisation :

a. Initialisation séquentielle

L'initialisation séquentielle permet de spécifier une valeur pour un ou plusieurs membres de la structure en suivant l'ordre de la définition.

Exemple : Reprenons l'exemple de la variable `etudiant1` déclarée précédemment. L'initialisation séquentielle peut se faire de la façon suivante :

```c
struct etudiant etudiant1 = {"Ye", "Afi", 'F', 28};
```

b. Initialisation sélective

Elle permet d'initialiser qu'un nombre de membres choisi. De ce fait, les membres doivent être désignés et précédés d'un point (`.`).

Exemple : Reprenons l'exemple de la variable `etudiant1` déclarée précédemment. L'initialisation sélective peut se faire de la façon suivante :

```c
struct etudiant etudiant1 = { .nom = "Ye", .prenom = "Afi" };
```

NB : Il est toutefois possible de mélanger les initialisations séquentielles et sélectives.

3. Accès aux membres d'une structure

L'accès à un membre d'une structure se réalise à l'aide de la variable de type structure et de l'opérateur suivi du nom du champ visé. Elle nous permet non seulement d'accéder à la valeur d'un membre, mais aussi de la modifier avec l'opérateur d'affectation.

Syntaxe :

```
<identificateur>.<nomDuMembre>;
```

Exemple :

```c
#include <stdio.h>

int main()
{
    struct etudiant {
        char* nom;
        char* prenom;
        char sexe;
        int age;
    };

    struct etudiant etudiant1;

    etudiant1.nom = "Ye";
    etudiant1.prenom = "Af";
    etudiant1.sexe = 'F';
    etudiant1.age = 28;

    printf("Nom = %s\n", etudiant1.nom);
    printf("Prenom = %s\n", etudiant1.prenom);
    printf("Sexe = %c\n", etudiant1.sexe);
    printf("Age = %d\n", etudiant1.age);

    return 0;
}
```

III. Définition de type avec `typedef`

Il est possible au programmeur de définir un type personnalisé. Le mot clé **`typedef`** est utilisé pour définir un nouveau nom pour un type existant ou déclaré. Il permet de simplifier ou de

clarifier l'écriture du code, en évitant de répéter des types complexes (Par exemple le mot clé **`struct`**) ou en donnant des noms plus explicites.

Exemple :

```c
#include <stdio.h>

int main()
{
    struct etudiant {
        char* nom;
        char* prenom;
        char sexe;
        int age;
    };

    typedef struct etudiant etudiant;

    etudiant etudiant1;

    etudiant1.nom = "Ye";
    etudiant1.prenom = "Af";
    etudiant1.sexe = 'F';
    etudiant1.age = 28;

    printf("Nom = %s\n", etudiant1.nom);
    printf("Prenom = %s\n", etudiant1.prenom);
    printf("Sexe = %c\n", etudiant1.sexe);
    printf("Age = %d\n", etudiant1.age);

    return 0;
}
```

Application :

Reprenons l'algorithme de l'application sur les tableaux (Chapitre 8). On souhaite à présent connaitre d'autres informations sur les étudiants comme leurs noms, prénom, etc

I. Notion d'énumération

Les énumérations en C sont un type de données qui permettent de définir un ensemble de constantes entières nommées. Elles sont utilisées pour améliorer la lisibilité et la maintenabilité du code en remplaçant des valeurs numériques par des noms significatifs.

II. Définition d'une énumération

```
1. enum <nomEnumeration> {
2.   <nomDuChamp>[=<valeur>],
3. };
```

Exemple :

```
1. enum jour {
2.     lundi,
3.     mardi,
4.     mercredi,
5.     jeudi,
6.     vendredi,
7.     samedi,
8.     dimanche
9. };
```

Remarque :

Dans l'exemple ci-dessous ; lundi vaut 0, mardi 1, à moins que des valeurs explicites ne soient attribuées.

III. Attribution de valeurs explicites

Il est possible d'attribuer des valeurs spécifiques aux constantes d'énumération. Si une valeur est attribuée à une constante, les constantes suivantes seront incrémentées automatiquement.

```
enum jour {
    lundi = 1,
    mardi,
    mercredi,
    jeudi,
    vendredi = 10,
    samedi,
    dimanche
};
```

IV. Déclaration et utilisation d'une variable de type énumération

1. Déclaration, affectation et affichage

```
#include <stdio.h>

int main()
{
  enum jour {
    lundi = 1,
    mardi,
    mercredi,
    jeudi,
    vendredi,
    samedi,
    dimanche
  };

  enum jour auj = samedi;

  printf("Aujourd'hui = %d\n", auj);

  return 0;
}
```

2. Utilisation dans une structure conditionnelle

Les énumérations sont souvent utilisées dans des instructions switch pour gérer différents cas de manière claire et concise.

Exemple :

```c
#include <stdio.h>

int main()
{
  enum jour {
    lundi = 1,
    mardi,
    mercredi,
    jeudi,
    vendredi,
    samedi,
    dimanche
  };

  enum jour auj = samedi;

  switch(auj) {
    case lundi:
        printf("Lundi\n");
        break;
    case samedi:
        printf("Samedi\n");
        break;
    case jeudi:
        printf("Jeudi\n");
        break;
    default:
        printf("Autre jour\n");
  }

  return 0;
}
```

V. Définition d'une Énumération avec typedef

En utilisant typedef, nous pouvons créer un alias pour un type énuméré, ce qui rend la déclaration de variables plus concise et lisible.

```
typedef enum {
    lundi,
    mardi,
    mercredi,
    jeudi,
    vendredi,
    samedi,
    dimanche
} Jour;
```

Le programme précédent devient :

```
#include <stdio.h>

int main()
{
  typedef enum jour {
    lundi = 1,
    mardi,
    mercredi,
    jeudi,
    vendredi,
    samedi,
    dimanche
  } Jour;

  Jour auj = samedi;

  switch(auj) {
    case lundi:
        printf("Lundi\n");
        break;
    case samedi:
        printf("Samedi\n");
        break;
    case jeudi:
        printf("Jeudi\n");
        break;
    default:
        printf("Autre jour\n");
  }

  return 0;
}
```

Une union est, à l'image d'une structure, un regroupement d'objet de type différents. La différence comparativement aux structures est qu'une union est un agrégat qui ne peut contenir qu'un seul de ses membres à la fois.

I. Définition d'une union

Une union est définie à l'aide du mot-clé union, similaire à une structure, mais tous ses membres partagent le même espace mémoire. Cela signifie qu'une union ne peut contenir qu'une seule valeur parmi ses membres à un moment donné.

1. Syntaxe de déclaration :

```
1. union <nomUnion> {
2.    <definition du membre 1>;
3.    <definition du membre 2>;
4.    <definition du membre n>;
5. };
```

L'identificateur `<nomUnion>` est appelé étiquette d'union

2. Exemple :

```
1. union nombre {
2.    int i;
3.    float f;
4. }
```

II. Différences clés entre une structure et une union

- Le mot-clé struct est utilisé pour définir une structure tandis que le mot-clé union est utilisée pour définir une union.
- Lorsqu'une variable est associée à une structure, le compilateur alloue la mémoire pour chaque membre. La taille de la structure est supérieure ou égale à la somme des tailles de ses membres. Alors dans le cas d'union lorsqu'une variable est associée à une union, le compilateur alloue la mémoire en considérant la taille la plus grande de la mémoire. Ainsi, la taille de l'union est égale à la taille du plus grand membre.

- Chaque membre d'une structure se voit attribuer une zone de stockage unique, tandis que la mémoire allouée est partagée par les membres individuels dans le cas de l'union.
- Dans Structure, la modification de la valeur d'un membre n'affectera pas les autres membres de la structure alors que l'union altérant la valeur de l'un des membres modifiera d'autres valeurs de membre.
- Dans la structure, chaque membre peut être consulté à la fois, tandis que dans les unions, un seul membre est accessible à la fois.
- Plusieurs membres d'une structure peuvent s'initialiser en une fois alors que dans les unions, seul le premier membre peut être initialisé.

Exemple : Qu'affiche le programme suivant ?

```c
#include <stdio.h>

int main()
{
  typedef union {
    int i;
    float f;
  } Data;

  // Déclaration d'une variable de type Data
  Data d;
  d.i = 42;
  d.f = 10;

  printf("i = %d\n", d.i);
  printf("f = %lf\n", d.f);
}
```

III. Utilisation d'une union dans une structure

```c
#include <stdio.h>

// Énumération pour indiquer le type de donnée dans l'union
typedef enum {
    ENTIER,
    FLOTTANT
} Type;

// Structure contenant l'union et un indicateur de type
typedef struct {
    Type type;
    union {
        int entier;
        float flottant;
    } valeur;
} Donnees;

int main() {
    // Création d'une variable de type Donnees
    Donnees d1, d2;

    // Stockage d'un entier dans l'union
    d1.type = ENTIER;
    d1.valeur.entier = 10;
    printf("Entier : %d\n", d1.valeur.entier);

    // Stockage d'un flottant dans l'union
    d2.type = FLOTTANT;
    d2.valeur.flottant = 3.14159;
    printf("Flottant : %f\n", d2.valeur.flottant);

    return 0;
}
```

I. Notions de pointeur

Un pointeur est une variable qui stocke l'adresse mémoire d'une autre variable. En d'autres termes, un pointeur "pointe" vers une zone spécifique de la mémoire où se trouve une valeur. Les pointeurs sont essentiels pour accéder et modifier directement les valeurs en mémoire, ce qui est particulièrement utile pour des opérations telles que la gestion de tableaux, la programmation orientée objet, et l'optimisation de la mémoire.

II. Déclaration de pointeurs

1. Syntaxe de déclaration d'un pointeur

```
<type de variable pointée> *<nom du pointeur>;
```

2. Exemple

Dans cet exemple, on déclare un pointeur `ptr` qu'on pointe sur la variable `x`. On modifie ensuite la valeur de `x` en utilisant le pointeur `ptr`.

```c
int x = 10;
int *ptr = &x;

printf("Valeur de x : %d\n", x); // Affiche 10
printf("Valeur pointée par ptr : %d\n", *ptr); // Affiche également 10

// Modification de la valeur via le pointeur
*ptr = 20;
printf("Nouvelle valeur de x : %d\n", x); // Affiche 20
```

III. Tableaux et Pointeurs

Les tableaux et les pointeurs sont étroitement liés en C. En effet, le nom d'un tableau n'est en réalité qu'un pointeur vers son premier élément.

```c
int tab[5] = {1, 2, 3, 4, 5};

// tab est un pointeur vers le premier élément
printf("Premier élément : %d\n", *tab); // Affiche 1

// Accès aux autres éléments via les pointeurs
printf("Deuxième élément : %d\n", *(tab + 1)); // Affiche 2
```

IV. Passage par adresse

En C, le passage par adresse est réalisé en utilisant des pointeurs. Lorsqu'une variable est passée par adresse à une fonction, l'adresse mémoire de cette variable est transmise à la fonction. Cela permet à la fonction de modifier directement la variable d'origine.

Exemples :

```c
#include <stdio.h>

void modifierVariable(int *ptr) {
  // Modification de la valeur pointée par ptr
  *ptr = 20;
}

int main() {
  int x = 10;

  printf("Avant modification : %d\n", x);

  // Passage de l'adresse de x à la fonction
  modifierVariable(&x);

  printf("Après modification : %d\n", x);

  return 0;
}
```

```c
#include <stdio.h>

void echange(int *a, int *b) {
  // Échange des valeurs
  int temp = *a;
  *a = *b;
  *b = temp;
}

int main() {
  int x = 10;
  int y = 20;

  printf("Avant échange : x = %d, y = %d\n", x, y);

  // Passage des adresses de x et y à la fonction echange
  echange(&x, &y);

  printf("Après échange : x = %d, y = %d\n", x, y);

  return 0;
}
```

I. Résumé des types en C

Désignation	Description	Valeur minimale	Valeur maximale	Format (Octet)
`_Bool`	Booléen	0	1	1
`signed char`	Type char signé	-127	127	1
`unsigned char`	Type char non signé	0	255	1

`short`
`short int`
`signed short`
`signed short int`
	Type entier minimum, court, entier et signé	-32768	32767	2

`unsigned short` `unsigned short int`	Type entier minimum, court	0	65536	2

`int`
`signed`
`signed int`
	Type entier standard, signé	-2147483648	2147483647	4

`unsigned` `unsigned int`	Type entier standard, non signé	0	4294967295	4

`long`
`longint`
`signed long`
`signed long int`
	Type entier long, entier et signé	-2^{63}	$2^{63}-1$	8

`unsigned long` `unsigned long int`	Type entier long, entier non signé	0	$2^{64}-1$	8

`long long`
`long long int`
`signed long long`
`signed long long int`
	Type entier long, entier et signé	-2^{63}	$2^{63}-1$	8

`unsigned long long`	Type entier long, entier non signé	0	$2^{64}-1$	8

`unsigned long long int`				
`float`	Type flottant. Simple précision			4
`double`	Type flottant. Double précision			8
`long double`	Type flottant. En pratique, selon le système, de la double précision à la quadruple précision.			16

II. Manipulation de caractères <ctype.h>

Les fonctions décrites ci-dessous permettent de tester une propriété du caractère passé en paramètre. Elles renvoient la valeur 1 si le caractère vérifie la propriété et `0` sinon.

Table 7 : Quelques fonctions de manipulation des caractères

Fonction	**Renvoie 1 si le caractère est**
`isalnum`	Une lettre ou un chiffre
`isalpha`	Une lettre
`iscntrl`	Un caractère de commande
`isdigit`	Un chiffre décimal
`isgraph`	Un caractère imprimable ou le blanc
`islower`	Une lettre minuscule
`isprint`	Un caractère imprimable (pas le blanc)
`ispunct`	Un caractère imprimable qui n'est ni une lettre ni un chiffre
`isspace`	Un blanc
`isupper`	Une lettre majuscule
`isxdigit`	Un chiffre hexadécimal

Nous disposons également des fonctions de conversions de caractères

Fonction	Prototype	Action
tolower	`int` **`tolower`**`(int c)`	Convertit **c** en minuscule si c'est une lettre majuscule, retourne **c** sinon.
toupper	`int` **`toupper`**`(int c)`	Convertit **c** en majuscule si c'est une lettre minuscule, retourne **c** sinon.

III. Fonction standard sur les chaînes de caractères dans <string.h>

Table 7: Quelques fonctions usuelles sur les chaines de caractères

Fonction	Action
`char *strcpy(char *ch1, char *ch2)`	Copie la chaîne ch2 dans la chaîne **ch1** ; retourne **ch1**.
`char *strcpy(char *ch1, char *ch2, int n)`	Copie **n** caractères de la chaîne **ch2** dans la chaîne **ch1** ; retourne **ch1**.
`char *strcat(char *ch1, char *ch2)`	Copie la chaîne **ch2** à la fin de la chaîne **ch1** ; retourne **ch1**.
`char *strncat(char *ch1, char *ch2, int n)`	Copie **n** caractères de la chaîne **ch2** à la fin de la chaîne **ch1** ; retourne **ch1**.
`int strcmp(char *ch1, char *ch2)`	Compare **ch1** et **ch2** pour l'ordre lexicographique ; retourne une valeur négative si **ch1** est inférieure à **ch2**, une valeur positive si **ch1** est supérieure à **ch2**, **0** si elles sont identiques.
`int strcmp(char *ch1, char *ch2, int n)`	Compare les **n** premiers caractères de **ch1** et **ch2**.
`char *strchr(char *chaine, char c)`	Retourne un pointeur sur la première occurrence de **c** dans **chaine**, et **NULL** si **c** n'y figure pas.
`char *strchr(char *chaine, char c)`	Retourne un pointeur sur la dernière occurrence de **c** dans **chaine**, et **NULL** si **c** n'y figure pas.
`char *strchr(char *ch1, char *ch2)`	Retourne un pointeur sur la première occurrence de **ch2** dans **ch1**, et **NULL** si **ch2** n'y figure pas.
`int strlen(char *chaine)`	Retourne la longueur de **chaine**.

IV. Résumé des opérateurs

Priorité	Opérateur	Description	Associativité
1	`++`	Incrémentation postfixée	Gauche-vers droite
	`--`	Décrémentation postfixée	
	`()`	Appel de fonction	
	`[]`	Indiçage de tableau	
	`.`	Sélection d'élément par référence	
	`->`	Sélection d'élément via un pointeur	
2	`++`	Incrémentation préfixée	Droite-vers-gauche
	`--`	Décrémentation préfixée	
	`+`	Plus unaire	
	`-`	Moins unaire	
	`!`	NON logique	
	`~`	NON bit à bit (complément à un)	
	`(type)`	Cast (conversion) de type	
	`*`	Indirection (déréférencement)	
	`&`	Adresse de	
	`Sizeof`	Taille en octets	
	`_Alignof`	Exigence d'alignement (depuis C11)	
3	`*`	Multiplication	Gauche-vers droite
	`/`	Division	
	`%`	Modulo (reste de la division)	
4	`+`	Addition	Gauche-vers droite
	`-`	Soustraction	
5	`<<`	Décalage de bits à gauche	
	`>>`	Décalage de bits à droite	
6	`<`	Inférieur à	
	`<=`	Inférieur ou égal à	
	`>`	Supérieur à	
	`>=`	Supérieur ou égal à	
7	`==`	Égal à	
	`!=`	Différent de	
8	`&`	ET bit à bit	

9	^	XOR bit à bit (OU exclusif)	
10	|	OU bit à bit (OU inclusif)	
11	&&	ET logique	
12	||	OU logique	
13	?:	Opérateur conditionnel ternaire	Droite-vers-gauche
14	=	Affectation directe	
	+=	Affectation par somme	
	-=	Affectation par différence	
	*=	Affectation par produit	
	/=	Affectation par quotient	
	%=	Affectation par reste	
	<<=	Affectation par décalage à gauche	
	>>=	Affectation par décalage à droite	
	&=	Affectation par ET bit à bit	
	^=	Affectation par XOR bit à bit	
	|=	Affectation par OU bit à bit	
15	,	Virgule	Gauche-vers droite